Complete RTL-to-GDSII ASIC Flow Workthrough and Debugging Journal

Purpose of this Document

This document is a complete walkthrough of the entire ASIC design flow executed during the 16-bit pipelined ALU project.

It documents:

- Every major design stage
- Commands that worked correctly
- Mistakes encountered
- Debugging process
- Tool behavior observations
- Important backend/frontend lessons
- Cadence tool issues and resolutions
- Best practices discovered during implementation

This serves as a practical engineering notebook for repeating the complete flow in future projects.

PHASE 1: RTL DESIGN IN VIVADO

Objective

Create a 16-bit pipelined ALU in Verilog supporting arithmetic and logical operations.

Initial RTL Architecture Planning

Design Decisions

Pipeline Architecture

The ALU was implemented using:

- Stage 1: Input pipeline registers
- Stage 2: ALU computation and output registers

This improved:

- timing
- synchronization
- backend synthesis quality

Inputs Defined

```
input clk, rst;
input [15:0] A;
input [15:0] B;
input [3:0] opcode;
input valid_in;
```

Outputs Defined

```
output reg [15:0] result;
output reg zero;
output reg carry;
output reg overflow;
output reg negative;
output reg valid_out;
```

Supported Operations

Opcode	Operation
0	ADD
1	SUB
2	MUL
3	DIV
4	AND
5	OR
6	XOR

Opcode	Operation
7	NOT
8	Shift Left
9	Shift Right
10	Compare Equal
11	Compare Greater

IMPORTANT RTL DESIGN LESSONS

Mistake #1

Incorrect Reset Logic Initially

Incorrect logic was accidentally written:

```
if(rst)
begin
    A_reg <= 16'd0;
end
else
begin
    A_reg <= A;
end
```

Later some assignments were accidentally reversed.

Lesson

Always verify:

- reset condition
- normal operating condition
- valid signal propagation

Mistake #2

Incorrect Overflow Logic for Subtraction

Initially:

```
alu_overflow = (A_reg[15] == B_reg[15]) && (A_reg[15] != alu_result[15]);
```

This was correct for ADDITION but incorrect for SUBTRACTION.

Correct Subtraction Overflow Logic

```
alu_overflow = (A_reg[15] != B_reg[15]) && (A_reg[15] != alu_result[15]);
```

Lesson

Addition and subtraction overflow conditions are DIFFERENT.

Mistake #3

Confusion Between '=' and '<='

Question encountered:

Why use:

```
=
```

inside combinational logic?

instead of:

```
<=
```

Resolution

Operator	Usage
=	Combinational logic
<=	Sequential logic

Final Rule

Use '=' inside:

```
always @(*)
```

Use '<=' inside:

```
always @(posedge clk)
```

IMPORTANT RTL IMPLEMENTATION DETAILS

Stage 1 Registers

```
reg [15:0] A_reg;
reg [15:0] B_reg;
reg [3:0] opcode_reg;
reg valid_stage1;
```

Internal ALU Signals

```
reg [15:0] alu_result;
reg alu_zero;
reg alu_carry;
reg alu_overflow;
reg alu_negative;
```

Stage 2 Registers

```
reg [15:0] result_reg;
reg zero_reg;
reg carry_reg;
reg overflow_reg;
reg negative_reg;
reg valid_stage2;
```

TESTBENCH DEVELOPMENT

Important Test Cases Added

Arithmetic Tests

- Addition
- Subtraction
- Multiplication
- Division
- Divide-by-zero

Logical Tests

- AND
- OR
- XOR
- NOT

Shift Tests

- Left Shift
- Right Shift

Comparison Tests

- Equal comparison
- Greater-than comparison

Corner Cases

- Overflow testing
- Negative outputs
- Zero outputs

IMPORTANT TESTBENCH LESSONS

Mistake #4

Pipeline Delay Confusion

Waveforms initially looked incorrect because:

- pipeline delay was not considered

- outputs appeared 1-2 cycles later

Lesson

Pipelined designs introduce intentional latency.

Outputs should NOT be expected immediately.

PHASE 2: XCELIUM / NCSIM SIMULATION

Initial Goal

Move RTL simulation from Vivado to Cadence simulation environment.

Major Problem Encountered

Xcelium GUI Crash

Symptoms

- SimVision opened
- signals displayed as XXXX
- simulation time stuck at 0
- popup:

```
XM-Sim terminated unexpectedly
```

Root Cause Analysis

The issue was NOT RTL-related.

The actual causes were:

- broken SimVision environment
- unstable GUI backend
- old Cadence installation
- X11/display issues

- waveform backend instability

IMPORTANT REALIZATION

Compilation and elaboration succeeded successfully.

Meaning:

```
RTL itself was already valid.
```

Working Xcelium Commands

Environment Setup

```
source cshrc_new
```

Compile + Run

```
xrun -R ALU.v tb_ALU.v
```

Commands That Caused Problems

Problematic GUI Invocation

```
xrun -gui ALU.v tb_ALU.v
```

Result

GUI/backend crash.

Important Simulation Lesson

Terminal-based simulation is often more stable than GUI simulation in older EDA environments.

NCSIM FLOW ATTEMPT

Commands Used

Compilation

```
ncvlog ALU.v tb_ALU.v
```

Elaboration

```
ncelab tb_ALU
```

Simulation

```
ncsim tb_ALU
```

Major Problem Encountered

Error

```
*F,NOWORK: no working library defined
```

Resolution

Create work directory

```
mkdir work
```

Create cds.lib

```
DEFINE work ./work
```

Create hdl.var

```
DEFINE WORK work
```

Lesson

Older Cadence flows require explicit work library definition.

FINAL DECISION

Return to Xcelium flow because:

- modern flow simpler
- RTL already validated
- GUI instability unrelated to design

PHASE 3: SYNTHESIS USING CADENCE GENUS

Objective

Convert RTL into gate-level standard-cell netlist.

Launch Command

```
genus &
```

Synthesis TCL Script

Working Script

```
read_hdl ALU.v

elaborate ALU

create_clock -period 10 clk

synthesize -to_mapped

report_area > area.rpt

report_timing > timing.rpt

report_power > power.rpt

write_hdl > ALU_synth.v

write_sdc > ALU.sdc
```

Important Synthesis Lessons

Mistake #5

Incomplete Timing Constraints

Only clock constraint was defined.

Missing:

- input delays
- output delays
- false paths
- multicycle paths
- MMMC setup

Result

Timing reports displayed:

```
Some unconstrained paths have not been displayed
```

Important Discovery

Genus inferred optimized arithmetic structures automatically.

Observed cells:

```
csa_tree_mul...
```

Meaning:

- optimized multiplier synthesis
- CSA tree inference
- intelligent arithmetic mapping

Synthesis Results

Area

```
Cell Count = 1100
Area = 8498.473
```

Power

Approximately:

```
2.4 mW
```

Cell Distribution

```
Sequential Cells = 57
Combinational Cells = 1043
```

Important Lesson

Multipliers significantly increase:

- area
- routing complexity
- power

PHASE 4: INNOVUS PHYSICAL DESIGN

Objective

Convert synthesized netlist into physical ASIC layout.

Launch Command

```
innovus &
```

Major Initial Mistake

Mistake #6

Innovus TCL commands were accidentally executed in Linux shell.

Example:

```
floorPlan
```

Result

```
command not found
```

Resolution

Innovus commands must ONLY be executed inside:

```
innovus>
```

prompt.

Initial Innovus Script

The first script failed because:

- no LEF files loaded
- no technology libraries loaded
- invalid site name

Major Errors Encountered

Error #1

```
Module DFFQX1 is not defined
```

Meaning

Innovus lacked physical standard-cell definitions.

Error #2

```
"core" does not match any site object
```

Meaning

Incorrect floorplan site name.

IMPORTANT BACKEND LESSON

Genus only needs:

- logical libraries

Innovus additionally requires:

- LEF files
- technology LEF
- physical geometry
- routing layers
- pin positions

PDK SEARCH PROCESS

Working Commands

```
find /opt/cadence -name "*.lef"
```

```
find /opt/cadence -name "*.lib"
```

Working PDK Files

Technology LEF

```
/opt/cadence/FOUNDRY/digital/45nm/dig/lef/gsclib045_tech.lef
```

Standard Cell LEF

```
/opt/cadence/FOUNDRY/digital/45nm/dig/lef/gsclib045_macro.lef
```

Timing Library

```
/opt/cadence/CONFRML211/share/cfm/lec/demo/rcv_intro/libs/gsclib045_v3.5/timing/
slow.lib
```

Final Working Innovus Script

```
set init_lef_file {
    /opt/cadence/FOUNDRY/digital/45nm/dig/lef/gsclib045_tech.lef
    /opt/cadence/FOUNDRY/digital/45nm/dig/lef/gsclib045_macro.lef
}

set init_lib_file {
```

```
    /opt/cadence/CONFRML211/share/cfm/lec/demo/rcv_intro/libs/gsclib045_v3.5/
timing/slow.lib
}

set init_verilog ALU_16_synth.v

set init_top_cell ALU

set init_sdc_file ALU.sdc

init_design

floorPlan -site CoreSite -r 1.0 0.7 20 20 20 20

place_design

ccopt_design

route_design

report_area > innovus_area.rpt

report_timing > innovus_timing.rpt

saveDesign ALU_innovus.enc
```

Important Innovus Lessons

Mistake #7

Timing setup incomplete.

Innovus warnings:

```
No constrained timing paths found
```

and:

```
No delay corners are defined
```

Consequence

- CTS partially failed
- timing-driven optimization unavailable
- physical-only routing completed

Important Realization

Even with incomplete timing setup:

- placement succeeded
- routing succeeded
- GDS generation succeeded

Meaning:

Physical implementation flow still worked.

PHYSICAL DESIGN RESULTS

Successfully achieved:

- floorplanning
- placement
- routing
- metal layer generation
- via insertion
- final layout generation

DRC and Connectivity Issues

verify_drc

Reported:

```
1000 violations
```

---

verifyConnectivity

Reported:

- opens
- disconnected IOs

Root Causes

- simplified academic flow
- incomplete IO constraints
- missing MMMC setup
- incomplete CTS optimization
- simplified physical constraints

FINAL GDS GENERATION

Working Command

```
streamOut final_ALU.gds -libName ALU
```

Successful Result

```
######Streamout is finished!
```

Meaning:

Final GDSII layout generation succeeded.

FINAL GENERATED FILES

File	Description
final_ALU.gds	Final physical layout
final_ALU.v	Final routed netlist

File	Description
final_ALU.enc	Innovus database
area.rpt	Area analysis
timing.rpt	Timing report
power.rpt	Power analysis

MOST IMPORTANT OVERALL LESSONS

Lesson #1

EDA tool issues are often unrelated to RTL quality.

Lesson #2

Successful compile/elaboration usually proves RTL correctness.

Lesson #3

Modern ASIC flow requires BOTH:

- logical libraries
- physical libraries

Lesson #4

Timing constraints are critical for:

- CTS
- optimization
- routing quality
- timing closure

Lesson #5

Backend implementation complexity increases dramatically after synthesis.

Lesson #6

Multipliers dominate:

- area
- routing congestion
- power

Lesson #7

Academic backend flows often simplify:

- IO planning
- MMMC
- CTS
- signoff verification

COMPLETE FLOW SUMMARY

```
RTL Design
    ↓
Functional Verification
    ↓
Cadence Simulation
    ↓
Genus Synthesis
    ↓
Area/Power Analysis
    ↓
Innovus Floorplanning
    ↓
Placement
    ↓
CTS
    ↓
Routing
    ↓
DRC/Connectivity
    ↓
GDSII Generation
```

CURRENT STATUS OF PROJECT

Stage	Status
RTL Design	Completed
Simulation	Completed
Synthesis	Completed
Area Analysis	Completed
Power Analysis	Completed
Floorplanning	Completed
Placement	Completed
Routing	Completed
GDSII Generation	Completed
Timing Closure	Partial
DRC Closure	Pending
Full CTS Optimization	Pending

NEXT FUTURE IMPROVEMENTS

Frontend

- CLA Adder
- Wallace Tree Multiplier
- Configurable ALU width
- Low-power RTL

Backend

- Full MMMC setup
- Proper SDC constraints
- CTS optimization
- DRC clean routing
- LVS verification
- STA using Tempus
- IR drop analysis
- congestion optimization

FINAL CONCLUSION

A complete practical RTL-to-GDSII ASIC flow was successfully executed for a 16-bit pipelined ALU using:

- Verilog RTL
- Cadence Xcelium
- Cadence Genus
- Cadence Innovus
- 45nm GPDK technology

The project provided hands-on exposure to:

- frontend RTL development
- synthesis
- physical design
- standard-cell mapping
- routing
- GDSII generation
- ASIC backend debugging

This work established a strong foundational understanding of modern digital ASIC implementation flow.